

On homeworks, we will often ask you to “**give an algorithm**” for a certain problem. Here is what you’ll need to provide, unless we explicitly give you an option, or say you don’t need to provide a certain component. (On exams we’ll tell you exactly what to provide.)

1. A brief but clear **high-level English description** of the algorithm’s structure, main steps, and what they do. If applicable, mention the design paradigm it follows (divide and conquer, dynamic programming, greedy, etc.).
2. **Precise pseudocode** for the algorithm and any subroutines. The emphasis should be on clarity and readability by humans (not “compilability” by a computer). A good programmer who reads your solution should be able to implement it in a real programming language without much trouble. You do not need to rewrite code for procedures we have already given, or that you have given before.
3. **Analysis** of your algorithm. This has two parts:

- **A correctness argument**, that your algorithm outputs a correct answer on any (valid) input. This often requires first showing some structural properties of the problem that your algorithm will exploit. (For example: the correctness argument for Euclid’s algorithm relies on the mathematical fact that $\gcd(x, y) = \gcd(y, x \bmod y)$ when $y \neq 0$. This is something we had to prove first.)

For recursive algorithms, correctness often relies implicitly on an inductive argument: that recursive calls (on smaller inputs) return correct answers, and that the algorithm properly combines those answers to get a correct answer for the original input. You do not need to present a formal induction proof, but your argument should be structured in a way that could be converted to one. See the last two sections of [Handout 2](#) for further details and an example.

- **A runtime analysis**, giving an asymptotic bound on the (worst-case) running time as a function of the input size. Usually a Big- O upper bound will suffice, unless we ask for a Big- Ω lower bound or a Big- Θ bound.

For a divide-and-conquer algorithm, you should typically first give a runtime recurrence and then solve it, using the Master Theorem or some other method (e.g., substitution).